

INFORMATICA – PYTHON

Clasa a IX-a | Fisa de lucru nr. 2

Metode de sortare in Python: Selectia minimului si Metoda bulelor

Competente specifice: CS 3.2, CS 4.2, CS 5.2, CS 6.2 | Domeniu: 2.4 Metode de sortare

1. De ce sortam datele?

Sortarea reprezinta aranjarea elementelor unei liste intr-o anumita ordine (crescatoare sau descrescatoare). Este una dintre cele mai frecvente operatii in programare – de la clasarea elevilor dupa medii, la ordonarea produselor dupa pret intr-un magazin online.

In aceasta fisa vom studia doua metode clasice de sortare:

- Sortarea prin selectia minimului (Selection Sort)
- Metoda bulelor (Bubble Sort)

2. Sortarea prin selectia minimului

2.1. Cum functioneaza?

Ideea de baza: la fiecare pas, cautam cel mai mic element din portiunea nesortata a listei si il punem la locul lui (la inceputul portiunii nesortate).

Pasi pentru o lista cu n elemente:

- Pas 1: gasim minimul din pozitiile 0, 1, ..., n-1 → il punem pe pozitia 0
- Pas 2: gasim minimul din pozitiile 1, 2, ..., n-1 → il punem pe pozitia 1
- ...
- Pas n-1: ultimele 2 elemente se compara si se sorteaza

2.2. Pseudocod

```
pentru i <- 0, n-2 executa
    poz_min <- i
    pentru j <- i+1, n-1 executa
        daca lista[j] < lista[poz_min] atunci
            poz_min <- j
    daca poz_min != i atunci
        interschimba lista[i] cu lista[poz_min]
```

2.3. Exemplu rezolvat pas cu pas

Lista initiala: [64, 25, 12, 22, 11]

Pas	Min gasit	Lista dupa interschimbare
i=0	11 (poz 4)	[11, 25, 12, 22, 64] ← 11 si 64 s-au schimbat
i=1	12 (poz 2)	[11, 12, 25, 22, 64] ← 12 si 25 s-au schimbat
i=2	22 (poz 3)	[11, 12, 22, 25, 64] ← 22 si 25 s-au schimbat
i=3	25 (poz 3)	[11, 12, 22, 25, 64] ← nicio schimbare
Final	–	[11, 12, 22, 25, 64] ✓ sortat!

2.4. Implementare Python

```
def selectia_minimului(lista):
    n = len(lista)
    for i in range(n - 1):
        poz_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[poz_min]:
                poz_min = j
        # Interschimbare
        if poz_min != i:
            lista[i], lista[poz_min] = lista[poz_min], lista[i]
    return lista

# Test
numere = [64, 25, 12, 22, 11]
print('Initial:', numere)
print('Sortat:', selectia_minimului(numere))
```

3. Metoda bulelor (Bubble Sort)

3.1. Cum functioneaza?

Ideea de baza: parcurgem lista in mod repetat si comparam elementele adiacente (vecine). Daca sunt in ordine gresita, le interschimbam. Elementele mari "buleaza" spre sfarsit, ca bulele de aer in apa.

3.2. Pseudocod

```
pentru i <- 0, n-2 executa
    s-a_facut_schimb <- Fals
    pentru j <- 0, n-2-i executa
        daca lista[j] > lista[j+1] atunci
            interschimba lista[j] cu lista[j+1]
            s-a_facut_schimb <- Adevarat
    daca s-a_facut_schimb = Fals atunci
        opreste (lista deja sortata)
```

3.3. Exemplu rezolvat pas cu pas

Lista initiala: [5, 3, 8, 1, 2]

Trecere	Comparari si interschimbari
i=0	[5,3] → 3<5 swap → [3,5,8,1,2] [5,8] → ok → [3,5,8,1,2] [8,1] → 1<8 swap → [3,5,1,8,2] [8,2] → 2<8 swap → [3,5,1,2,8]
i=1	[3,5] → ok [5,1] → swap → [3,1,5,2,8] [5,2] → swap → [3,1,2,5,8]
i=2	[3,1] → swap → [1,3,2,5,8] [3,2] → swap → [1,2,3,5,8]
i=3	[1,2] → ok → nicio schimbare → oprire
Final	[1, 2, 3, 5, 8] ✓ sortat!

3.4. Implementare Python

```
def metoda_bulelor(lista):
    n = len(lista)
    for i in range(n - 1):
        s_a_facut_schimb = False
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                s_a_facut_schimb = True
        # Optimizare: daca n-a fost nicio schimbare, lista e sortata
        if not s_a_facut_schimb:
            break
    return lista

# Test
numere = [5, 3, 8, 1, 2]
print('Initial:', numere)
print('Sortat:', metoda_bulelor(numere))
```

4. Comparatie intre cele doua metode

Criteriu	Selectia minimului	Metoda bulelor
Nr. comparatii	Intotdeauna $n*(n-1)/2$	Max $n*(n-1)/2$, poate fi mai mic
Nr. interschimbări	Cel mult $n-1$	Poate fi mai mare
Optimizare posibila	Nu	Da (daca nu s-au facut schimburi)
Cel mai potrivit pentru	Cand interschimbările sunt costisitoare	Cand lista este aproape sortata

5. Exerciții

Exercițiul 1 – Urmărește algoritmul

Aplica manual selectia minimului pe lista: [7, 2, 9, 4, 1, 6]

Completeaza tabelul de mai jos:

Pas i	Min gasit la poz.	Lista dupa pas
i=0		
i=1		
i=2		
i=3		
i=4		

Exercițiul 2 – Cod Python cu erori

Codul de mai jos contine 3 greseli. Gaseste-le si corecteaza-le:

```
def selectia_minimului(lista):
    n = len(lista)
    for i in range(n):
        poz_min = 0
        for j in range(i + 1, n):
            if lista[j] < lista[poz_min]:
                poz_min = j
        lista[i] = lista[poz_min]
```

Eroarea 1: _____

Eroarea 2: _____

Eroarea 3: _____

Exercitiul 3 – Problema aplicata

Un profesor are notele clasei: [7, 9, 6, 10, 5, 8, 7, 4, 9, 6]

Scrie un program Python care:

- sorteaza notele in ordine crescatoare folosind metoda bulelor
- afiseaza nota minima, maxima si media
- afiseaza cati elevi au nota sub 5

Codul tau: